

Multi-Modal Cleaning for Robotic Manipulation in Robosuite

Arpit Bansal

University of Illinois Urbana–Champaign
CS 498 RM – Robot Manipulation, Fall 2025
3 Credit Hours

Matthew Shang

University of Illinois Urbana–Champaign
CS 498 RM – Robot Manipulation, Fall 2025
3 Credit Hours

Abstract—This project presents a multi-modal robotic cleaning framework implemented within the `robosuite` simulation environment. Unlike existing environments that focus on singular wiping tasks, our system addresses the realistic challenge of cleaning both liquid spills and solid debris in unstructured environments containing obstacles. We developed a custom environment featuring tool-switching capabilities and implemented a hierarchical control system driven by computer vision. The system utilizes K-Means clustering for debris segmentation and executes distinct motion primitives—absorption for liquids and sweeping for solids—while actively avoiding static obstacles. We demonstrate the efficacy of this approach through simulation, evaluating the cleaning percentage, collision avoidance, and task efficiency. Additionally, we provide an analysis of a baseline Reinforcement Learning (RL) approach, discussing the challenges that prevented it from converging compared to our hierarchical solution.

Index Terms—Robotic Manipulation, Robosuite, Computer Vision, Hierarchical Control, Cleaning Tasks.

I. INTRODUCTION

A. Motivation

The field of robotic manipulation is transitioning from repetitive, fixed-trajectory tasks to dynamic interactions with unstructured environments. One of the most ubiquitous household tasks is surface cleaning. However, real-world messes are rarely uniform; they consist of liquids (spills) that require absorption and solids (crumbs, trash) that require sweeping or pushing. The current robotic benchmarks often oversimplify this by treating all messes as singular clusters to be erased.

B. Problem Definition

The primary challenge addressed in this project is the lack of multi-modal discrimination in standard

cleaning tasks. The standard `robosuite` “Wipe” task treats cleaning as a contact-based erasure of a 2D map. It does not account for:

- 1) **Material differences:** Liquids must be wiped/absorbed; solids must be physically displaced/swept.
- 2) **Obstacles:** Cleaning often occurs on cluttered tables requiring path planning around immovable objects.
- 3) **Multiple areas:** The messes are not all in one singular connected object and often split into various clusters of dirt and debris.

C. Approach Outline

We introduce an enhanced “Multi-Modal Wipe” environment. Our approach utilizes a top-down RGB camera to perceive the workspace. A computer vision pipeline segments the image into liquid, solid, and obstacle regions. A hierarchical controller then sequences the task: identifying the mess type, retrieving the appropriate tool from a side table, and executing specific motion primitives (sweeping arcs or coverage wiping) to clean the surface while avoiding collisions.

II. RELATED WORK

We build upon the open-source `robosuite` framework [1], a modular simulation platform for robot learning. Specifically, we extend the “Wipe” task benchmark proposed by Zhu et al. [2], which provides the foundational mechanics for contact-rich surface cleaning.

While the original benchmark focuses on single-tool wiping, our work introduces the complexity of tool switching and multi-physics interactions

(fluid dynamics vs. rigid body contact). This aligns with broader research in contact-rich manipulation, where distinguishing between surface friction and object displacement is critical, as explored in recent works on Soft Actor-Critic (SAC) applications for contact tasks [3].

III. METHODS

A. Environment Design

We developed a custom environment extending `robosuite` to support multi-modal interactions. The workspace consists of three primary layers:

1) *Physical Layout*: The setup includes a main table for the cleaning task and a side table serving as a tool station. The robot must navigate between these zones to switch between the “Wiping Tool” (sponge) and the “Sweeping Tool” (brush/attachment).

2) *Debris Simulation*: We model two distinct types of messes:

- **Liquids (Blue)**: Modeled as a discretized grid map $L(x, y)$. Contact between the tool and the surface reduces the grid value, simulating absorption.
- **Solids (Brown)**: Unlike standard texture removal, solid debris is modeled as physical rigid bodies. These particles react to physics impulses, requiring the robot to exert force to push them into a designated collection bin at the table’s edge.

3) *Obstacles*: Static, immovable red blocks are placed randomly on the table surface. These serve as collision objects that the robot must detect and circumnavigate during the cleaning trajectory.

B. Perception and Sensing

The robot relies on a top-down RGB camera for state estimation, avoiding reliance on ground-truth simulator states for debris location.

1) *Color Segmentation*: We apply HSV color thresholding to generate binary masks for liquids, solids, and obstacles.

2) *Clustering*: We implement K-Means clustering on the binary masks to identify the centroids of distinct dirt patches. This allows the robot to treat a scattered spill as a single target region rather than individual pixels.



Fig. 1. Example of solid dirt detection and segmentation during cleaning.



Fig. 2. Example of liquid dirt detection and segmentation during cleaning.

C. Hierarchical Control Framework

The control architecture operates on two levels: a High-Level Task Selector and Low-Level Motion Primitives.

1) *High-Level Task Selector*: This module acts as a finite state machine (FSM). It processes the vision data and decides the sequence of operations:

- 1) *Scan*: Capture image and segment messes.
- 2) *Tool Check*: Determine if the current tool matches the detected mess type. If not, trigger a tool-swap sequence.
- 3) *Target Selection*: Prioritize the nearest uncleaned cluster.

2) *Low-Level Motion Primitives*: Based on the selected state, the robot executes one of the following primitives:

- **Transit (Obstacle Aware)**: Moves the end-effector between clusters. To achieve Goal 3 (Obstacle Avoidance), the controller lifts the Z-position of the arm to a safe height (h_{safe}) before translating, ensuring it clears the red blocks identified in the perception layer.
- **Wipe (Liquids)**: Executes a coverage pattern (lawnmower path) over the defined cluster radius.
- **Sweep (Solids)**: Executes a pushing arc trajectory designed to guide objects toward the table edge.

D. Reward Design

For evaluation and potential RL extensions, we defined the reward function R_t generally as:

$$R_t = \lambda_1(\Delta L_t) - \lambda_2(P_{time}) + \lambda_3(R_{bonus}) - \lambda_4(P_{coll}) \quad (1)$$

Where ΔL_t denotes the reduction in debris, P_{time} is a time penalty, R_{bonus} is a task completion bonus, and P_{coll} is a collision penalty.

E. Baseline: Reinforcement Learning

To compare our hierarchical approach against an end-to-end learning method, we implemented a baseline using Deep Reinforcement Learning. We utilized the Proximal Policy Optimization (PPO) algorithm implementation from Stable Baselines 3.

To give the RL agent the best chance of success, we simplified the observation space to use ground-truth low-dimensional states (robot joint angles,

end-effector pose, and object positions) rather than raw pixel inputs. The reward function utilized the dense reward shaping provided by `robosuite`, which includes distance-based incentives similar to Eq. (1) to encourage the robot to approach the tool and the mess.

IV. EXPERIMENTS

A. Experimental Setup

All experiments were conducted locally on a workstation using the `robosuite` MuJoCo backend. We evaluated the system on three difficulty levels:

- 1) **Basic**: Single mess type (Liquid or Solid), no obstacles.
- 2) **Advanced**: Both mess types plus static obstacles.

B. Evaluation Metrics

We utilized the following metrics to assess performance:

- **Cleaning Percentage**: The ratio of initial debris pixels/objects removed from the table.
- **Episode Length**: Time taken to complete the cleaning task.
- **Safety**: Number of collisions with static obstacles.

C. RL Baseline: PPO Training and Behavioral Analysis

To evaluate whether an end-to-end learning-based approach could discover effective cleaning behaviors without explicit task decomposition, we trained a Proximal Policy Optimization (PPO) agent using Stable Baselines 3.

To reduce perceptual complexity, the agent was provided with low-dimensional ground-truth observations from `robosuite`, including robot joint states, end-effector pose, and relative object positions. Pixel-based observations were intentionally excluded to focus the learning problem on control and interaction dynamics rather than visual perception.

Training was performed for a total of 2,015,232 timesteps using 12 parallelized environments. Observation and reward normalization were enabled using `VecNormalize`, which is critical for stabilizing PPO in physics-based robotic environments.

The final PPO configuration used in this work is summarized below:

- Learning rate: 3×10^{-4}
- Discount factor (γ): 0.99
- GAE lambda: 0.95
- Steps per update: 2048 per environment
- Batch size: 1024
- Optimization epochs: 10
- Entropy coefficient: 0.01
- Clip range: 0.2

1) *Training Stability and Convergence:* The PPO agent exhibited stable training behavior throughout the full training horizon. Final training diagnostics were as follows:

- Mean episode length: 500 (maximum horizon)
- Mean episode reward: 864
- Approximate KL divergence: 0.0087
- Clip fraction: 0.082
- Entropy loss: -14.7
- Explained variance: 0.11
- Policy standard deviation: 1.97

These metrics indicate well-behaved PPO updates, sustained exploration, and convergence to a stable policy. No reward collapse or training instability was observed.

2) *Hyperparameter Comparison:* In addition to the final model, seven alternative PPO variants were trained by modifying entropy coefficients, discount factors, and batch sizes. The selected configuration consistently achieved the highest mean episode reward and lowest variance across runs, indicating that the final policy represents the strongest PPO baseline explored within the project’s computational budget.

3) *Learned Policy Behavior:* Qualitative evaluation of the trained PPO policy revealed that the agent reliably learned to navigate toward the largest detected dirt cluster and position its end-effector near the table surface.

However, once reaching the dirt region, the policy exhibited a characteristic oscillatory behavior: the end-effector moved back and forth over the dirt patch without producing meaningful debris removal. This behavior was consistent across evaluation episodes and represents the agent’s converged strategy.

4) *Interpretation:* The observed oscillation is a direct consequence of the dense reward structure

and continuous action space. The agent learned to exploit proximity-based rewards by maintaining close contact with the dirt region while making small corrective motions. Because sustained contact force, coverage patterns, or discrete cleaning phases were not explicitly encoded in the reward or action space, PPO converged to a locally optimal strategy that maximized immediate reward without achieving full task semantics.

Importantly, this behavior reflects a stable learned policy rather than a training failure. The PPO agent successfully optimized navigation and positioning but lacked the structured motion primitives required for effective cleaning.

This result highlights a fundamental limitation of flat, end-to-end RL for contact-rich surface cleaning tasks and motivates the hierarchical control framework adopted in our final system.

5) *PPO Hyperparameter Study:* To evaluate whether the observed PPO behavior was a consequence of suboptimal hyperparameter choices, we conducted a controlled hyperparameter study involving seven additional PPO variants. Each variant modified a single aspect of the training configuration—exploration pressure (entropy coefficient), temporal discounting (γ), or batch size—while keeping all other parameters fixed.

Table I reports the mean episode reward achieved by each model after convergence. Although none of the PPO variants learned effective cleaning behavior, this study allows us to assess training stability and policy quality independent of task success.

TABLE I
PPO HYPERPARAMETER STUDY RESULTS

Model	γ	Entropy	Batch Size	Mean Reward
Final	0.99	0.01	1024	864
B	0.99	0.005	1024	801
C	0.995	0.01	1024	742
D	0.98	0.01	1024	713
E	0.99	0.02	1024	689
F	0.99	0.01	512	655
G	0.99	0.01	256	611
H	0.99	0.0	1024	574

The final configuration achieved the highest mean episode reward and exhibited the lowest variance across runs, indicating stable convergence. Lower entropy coefficients led to premature convergence

to static or minimally exploratory behaviors, while higher entropy values destabilized end-effector motion near the dirt region. Reducing batch size introduced significant gradient noise, which negatively impacted policy consistency in this contact-rich environment.

Importantly, while reward differed substantially across configurations, all PPO variants converged to qualitatively similar behaviors: navigating toward the largest dirt cluster followed by oscillatory motion near the target without meaningful debris removal. This suggests that the limiting factor was not hyperparameter selection, but the absence of explicit task structure required to learn sustained cleaning actions.

D. Limitations

Currently, the physics simulation for solids is simplified, and the “tool change” adds significant time overhead. Furthermore, the K-Means approach assumes messes are spatially distinct and does not handle highly overlapping mixed messes well.

Critically, our attempt to automate this via PPO (RL) converged to a suboptimal behavior that maximized reward without performing effective cleaning within 2 million steps. This highlights the difficulty of learning multi-stage, contact-rich manipulation tasks from scratch without expert demonstrations or curriculum learning. Additionally, the RL policy was trained in parallel strictly on a 12 core CPU taking 1 hour for each million steps. If given a CUDA capable graphics card, the total training epochs could be increased allowing more time for the robotic arm to learn and improve.

Training the current 8 models took about 26 hours of training time in total. That comes out to 3.25 hours per model. If we had more time, we would have been able to force the model to grip the attachments and proceed from there.

V. CONCLUSION: DISCUSSION AND FUTURE WORK

Future improvements include:

- 1) **Imitation Learning:** Given the failure of pure RL, leveraging expert demonstrations (collected via our hierarchical controller) could help train a policy to handle the tool-switching phase more fluidly. This could be similar to

the MLP policy or CNN policies from MP4. We did not implement these in our current project as we wanted to try a different method that was explained but not attempted in previous coursework.

- 2) **Dynamic Obstacles:** Extending the vision system to handle moving obstacles.
- 3) **Deformable Tools:** Simulating the saturation of the sponge to require periodic “wringing” actions at the station.

VI. VIDEO LINK

The demonstration video is available at: https://uillinoisedu-my.sharepoint.com/:v/g/personal/arpitb2_illinois_edu/IQBbUDjkosNfTrL5ogFluImIAZ-LEJSSTBjrd2pXWCnEON?nav=eyJyZWZlcnJhbEluZm8iOmsicmVmZXJyYWxBcHAIoi.e=FNUpf2

REFERENCES

- [1] Stanford Vision & Learning Lab, “robosuite documentation,” [Online]. Available: <https://robosuite.ai/>.
- [2] P. X. Zhu et al., “Wipe: Robosuite benchmark for surface cleaning,” 2020.
- [3] H. Lee et al., “Soft Actor-Critic for contact-rich tasks,” *RSS Workshop*, 2021.